

# HUNT EYE

Autonomous UAV Target Detection, Tracking & Interception

3-Person Team — Phase-by-Phase Implementation Plan

|        |       |       |      |
|--------|-------|-------|------|
| 6      | 15    | 16    | 3    |
| Phases | Steps | Weeks | Devs |

|                                            |                                        |                                               |
|--------------------------------------------|----------------------------------------|-----------------------------------------------|
| Dev 1                                      | Dev 2                                  | Dev 3                                         |
| Infra & Systems                            | Vision & Tracking                      | AI, Control & Safety                          |
| HAL · Orchestrator Dashboard · Sim-to-Real | YOLOv8 · MiDaS · ByteTrack OSNet Re-ID | MAVLink · PID · PPO A* Planner · Safety Layer |

Python Powered · RTX 3050 · Windows 11 + WSL2 · 100% Open Source

# Team Roles & Ownership

Each developer owns one full domain end-to-end. Nobody waits on someone else mid-phase. Shared infrastructure (env setup, folder skeleton) is done together in Phases 0–1, then the team splits.

Dev 1

Infra & Systems

- Phase 0 — Environment setup (WSL2, CUDA, AirSim, PX4, venv)
- Phase 1 — Folder skeleton, config.yaml, SharedState, HAL, Orchestrator
- Phase 1 — Dashboard (PyQt6 live window)
- Phase 6 — Sim-to-real switch, RealBackend, serial MAVLink

Steps: 0A, 0B, 0C, 0D, 0E, 1, 2, 3, 15

Dev 2

Vision & Tracking

- Phase 2 — YOLOv8 detector, MiDaS depth estimator, Terrain cost map
- Phase 3 — ByteTrack multi-object tracker, TargetFilter lock-on
- Phase 6 — OSNet Re-ID engine (reacquisition after occlusion)

Steps: 4, 5, 6, 7, 8, 13

Dev 3

AI, Control & Safety

- Phase 4 — MAVLink interface, PID controller
- Phase 5 — Gymnasium env wrapper, PPO training, A\* path planner
- Phase 6 — Safety layer (velocity cap, geofence, emergency land)

Steps: 9, 10, 11, 12, 14

# Master Timeline

| Phase | Weeks | Dev 1 — Infra                             | Dev 2 — Vision                | Dev 3 — AI & Control        |
|-------|-------|-------------------------------------------|-------------------------------|-----------------------------|
| Ph 0  | 1–2   | WSL2, CUDA, AirSim, PX4, venv             | Same — all 3 setup together   | Same — all 3 setup together |
| Ph 1  | 3–4   | SharedState, HAL, Orchestrator, Dashboard | Review & test camera feed     | Review & test camera feed   |
| Ph 2  | 5–7   | —                                         | YOLOv8, MiDaS, Terrain loader | —                           |
| Ph 3  | 8–9   | —                                         | ByteTrack, TargetFilter       | —                           |
| Ph 4  | 10–11 | —                                         | —                             | MAVLink, PID controller     |
| Ph 5  | 12–14 | —                                         | —                             | Gym env, PPO training, A*   |
| Ph 6  | 15–16 | Sim-to-real, RealBackend                  | OSNet Re-ID                   | Safety layer                |

|                          |                          |           |                         |
|--------------------------|--------------------------|-----------|-------------------------|
| <b>PHASE</b><br><b>0</b> | <b>Environment Setup</b> | Weeks 1–2 | ALL THREE DEVS together |
|--------------------------|--------------------------|-----------|-------------------------|

This is the only phase where all three work on the same steps. Everyone must have a working environment before splitting. Do this in one sitting together if possible.

|    |       |                                                      |         |
|----|-------|------------------------------------------------------|---------|
| 0A | Dev 1 | Install WSL2 (Windows Subsystem for Linux)           | ~1 hr   |
| 0B | Dev 1 | Install Python 3.10 + CUDA 11.8 + verify GPU         | ~1 hr   |
| 0C | Dev 1 | Install AirSim binary — drone takes off test         | ~30 min |
| 0D | Dev 1 | Install PX4 SITL inside WSL2                         | ~2 hrs  |
| 0E | Dev 1 | Create Python virtual environment + all pip installs | ~30 min |

Step 0B — GPU Verify Commands (all three run these):

```
python --version # Python 3.10.x
nvcc --version # CUDA 11.8
python -c "import torch; print(torch.cuda.is_available())" # True
pip install torch torchvision torchaudio \
--index-url https://download.pytorch.org/whl/cu118
```

Step 0C — AirSim Connection Test:

```
pip install airsims
import airsims
client = airsims.MulticopterClient()
client.confirmConnection()
client.enableApiControl(True)
client.armDisarm(True)
client.takeoffAsync().join()
print("OK")
```

Step 0E — Full Package Install:

```
pip install airsims ultralytics opencv-python timm torch torchvision \
stable-baselines3 gymnasium pymavlink PyQt6 numpy geopy \
osmnx torchreid rasterio tensorboard
# ByteTrack:
git clone https://github.com/ifzhang/ByteTrack.git
cd ByteTrack && pip install -r requirements.txt && pip install -e .
```

■ MILESTONE

NE

All 3 devs: AirSim open, drone hovering, torch.cuda = True. Workshop ready.

|                          |                         |           |                          |
|--------------------------|-------------------------|-----------|--------------------------|
| <b>PHASE</b><br><b>1</b> | <b>Project Skeleton</b> | Weeks 3–4 | Dev 1 leads · all review |
|--------------------------|-------------------------|-----------|--------------------------|

Dev 1 builds the entire scaffold. Dev 2 and Dev 3 read and understand it — they will build on top of it from Phase 2 onwards. This is the most critical phase for team alignment.

|    |       |                                                    |        |
|----|-------|----------------------------------------------------|--------|
| 1  | Dev 1 | Create folder structure + config.yaml              | ~1 hr  |
| 2  | Dev 1 | SharedState dataclass + threading lock             | ~2 hrs |
| 3  | Dev 1 | HAL (Hardware Abstraction Layer) — sim/real switch | ~2 hrs |
| 3b | Dev 1 | SystemOrchestrator — starts all threads            | ~2 hrs |
| 3c | Dev 1 | CameraStream (30 Hz) + PyQt6 Dashboard             | ~3 hrs |

config.yaml — Central Configuration (critical shared file):

```
mode: sim # Dev 1 changes to "real" in Phase 6
yolo:
model: "yolov8n.pt"
confidence: 0.45
target_classes: ["person"]
safety:
max_velocity_ms: 5.0
max_altitude_m: 30.0
geofence_radius_m: 100.0
mavlink:
connection: "udp:127.0.0.1:14550"
```

SharedState — Central Data Bus (every dev reads/writes this):

```
@dataclass
class SharedState:
    lock: threading.RLock
    latest_frame: np.ndarray = None # Dev 1 writes, Dev 2 reads
    detections: list = [] # Dev 2 writes, Dev 3 reads
    target_bbox: tuple = None # Dev 2 writes, Dev 3 reads
    active_target_id: int = -1 # Dev 2 writes
    cost_map: np.ndarray = None # Dev 2 writes, Dev 3 reads
    planned_path: list = [] # Dev 3 writes, Dev 1 displays
    system_mode: str = "idle" # Dev 3 writes, everyone reads
    uav_position: np.ndarray = None # Dev 1 (HAL) writes
```

Dev 2 & Dev 3 task during Phase 1: read every file Dev 1 creates. Understand SharedState fields. Agree on the interface before coding starts.

|                                                   |                                                                            |
|---------------------------------------------------|----------------------------------------------------------------------------|
| <div> <div>■ MILESTONE</div> <div>NE</div> </div> | python main.py — drone hovers, PyQt6 window shows live AirSim camera feed. |
|---------------------------------------------------|----------------------------------------------------------------------------|

## PHASE 2

## Vision — Making the Drone See

Weeks 5–7

Dev 2 owns this entirely

Dev 1 is polishing Orchestrator and dashboard. Dev 3 is reading RL papers and Gymnasium docs. Dev 2 builds all three vision modules — they integrate via SharedState.

|   |       |                                                              |        |
|---|-------|--------------------------------------------------------------|--------|
| 4 | Dev 2 | YOLOv8n object detector wrapper — writes to state.detections | ~2 hrs |
| 5 | Dev 2 | MiDaS depth estimator — writes to state.depth_map            | ~2 hrs |
| 6 | Dev 2 | Terrain loader (GDAL + osmnx) — writes to state.cost_map     | ~4 hrs |

YOLOv8 Detector — perception/detector.py:

```
from ultralytics import YOLO
class Detector:
    def __init__(self, config, state):
        self.model = YOLO(config["model"]) # auto-downloads yolov8n.pt
        self.conf = config["confidence"]
        self.target_classes = config["target_classes"]
        self.state = state
    def detect(self, frame):
        results = self.model(frame, conf=self.conf, verbose=False)
        dets = []
        for r in results:
            for box in r.bboxes:
                if self.model.names[int(box.cls)] in self.target_classes:
                    dets.append({"bbox": box.xyxy[0].tolist(),
                                "confidence": float(box.conf)})
        with self.state.lock:
            self.state.detections = dets # Dev 3 reads this
        return dets
```

MiDaS Depth Estimator — perception/depth\_estimator.py:

```
self.model = torch.hub.load("intel-isl/MiDaS", "MiDaS_small")
self.model.cuda() # runs on RTX 3050
# Output: 2D numpy array – same shape as frame
# bright pixel = far, dark pixel = near
with self.state.lock:
    self.state.depth_map = depth_np
```

Terrain Loader — simulation/terrain\_loader.py:

```
# config.yaml terrain.bbox = [72.8, 18.9, 72.95, 19.05] (Mumbai)
buildings = ox.features_from_bbox(..., tags={"building": True})
slope_cost = np.sqrt(gradient[0]**2 + gradient[1]**2) * 10
cost_map = np.clip(slope_cost + obstacle_map * 100, 0, 100)
# 0 = open air | 100 = solid obstacle
with self.state.lock:
    self.state.cost_map = cost_map # Dev 3 A* reads this
```

### ■ MILESTO NE

Dashboard shows bounding boxes on people + depth map overlay + coloured cost map.

**PHASE**  
**3**

## Tracking — Locking On

Weeks 8–9

Dev 2 continues

Dev 2 extends the vision pipeline with tracking. Dev 3 starts building the Gymnasium environment wrapper in parallel — they do not depend on each other yet.

|   |              |                                                                      |        |
|---|--------------|----------------------------------------------------------------------|--------|
| 7 | <b>Dev 2</b> | ByteTrack multi-object tracker — wraps detection into persistent IDs | ~3 hrs |
| 8 | <b>Dev 2</b> | TargetFilter — picks one target, triggers Re-ID on loss              | ~2 hrs |

ByteTrack Wrapper — `tracking/tracker.py`:

```
self.tracker = BYTETracker(
    track_thresh = 0.5, # min confidence to start tracking
    track_buffer = 30, # frames before dropping lost track
    match_thresh = 0.8 # IoU threshold for frame-to-frame match
)
# Input: detection boxes from state.detections
# Output: list of Track objects, each with .track_id and .tlwh
```

TargetFilter — `tracking/target_filter.py`:

```
if active_id == -1: # No lock yet
    best = max(tracks, key=lambda t: t.score)
    state.active_target_id = best.track_id
    state.target_bbox = best.tlwh # Dev 3 PID reads this
else:
    matched = [t for t in tracks if t.track_id == active_id]
    if matched:
        state.target_bbox = matched[0].tlwh
    else:
        state.system_mode = "reacquiring" # triggers Dev 2 OSNet in Ph 6
```

**■ MILESTONE**  
**NE**

One target tracked with persistent ID — doesn't change when others enter frame.

## PHASE 4

## Control — Making It Move

Weeks 10–11

Dev 3 owns this

Dev 2 is polishing tracker edge cases. Dev 1 is helping integrate dashboard with tracker output. Dev 3 implements MAVLink and the PID controller — the drone starts physically chasing the target.

9    **Dev 3**    MAVLink interface + PID controller — control/mavlink\_interface.py

~4 hrs

### PID Controller Logic:

```
# Tune these – start with these defaults:
self.Kp = 0.002 # proportional: main correction strength
self.Ki = 0.0001 # integral: removes persistent drift
self.Kd = 0.001 # derivative: prevents overshoot
# Each control loop iteration (20 Hz):
error_x = target_cx - frame_cx # + = target right of centre
error_y = target_cy - frame_cy # + = target below centre
vx = Kp*error_x + Ki*integral_x + Kd*(error_x - prev_error_x)
vy = Kp*error_y + Ki*integral_y + Kd*(error_y - prev_error_y)
vx = max(-2.0, min(2.0, vx)) # hard cap 2 m/s for now
self._send_velocity(vx, vy, -0.1, 0)
```

### MAVLink Connection:

```
from pymavlink import mavutil
self.conn = mavutil.mavlink_connection("udp:127.0.0.1:14550")
self.conn.wait_heartbeat() # blocks until PX4 SITL responds
# In Phase 6, Dev 1 swaps this to: "/dev/ttyUSB0" for real drone
```

### PID Tuning Guide:

| Symptom                         | Fix         |
|---------------------------------|-------------|
| Drone oscillates                | Reduce Kp   |
| Drone drifts without correcting | Increase Ki |
| Drone overshoots target         | Increase Kd |

### ■ MILESTONE

Drone moves to keep target centred. Target walks left — drone follows. PID tuned.

## PHASE 5

## AI Brain

Weeks 12–14

Dev 3 continues

The hardest phase technically. Dev 3 replaces the PID with a learned RL policy and adds the A\* path planner. Dev 2 starts OSNet Re-ID work. Dev 1 monitors training logs and helps debug reward shaping.

|    |       |                                                         |        |
|----|-------|---------------------------------------------------------|--------|
| 10 | Dev 3 | Gymnasium env wrapper — observation, action, reward     | ~4 hrs |
| 11 | Dev 3 | PPO training run — leave overnight, monitor TensorBoard | 8+ hrs |
| 12 | Dev 3 | A* path planner on cost map — 8-directional grid        | ~3 hrs |

Gymnasium Environment — `ai/gym_env.py`:

```
# What the RL agent SEES (8 numbers):
observation_space = gym.spaces.Box(low=-1, high=1, shape=(8,))
# [norm_target_cx, norm_target_cy, norm_bbox_area,
# drone_vx, drone_vy, drone_vz, drone_alt, target_lost_flag]
# What the RL agent DOES (4 numbers):
action_space = gym.spaces.Box(low=-1, high=1, shape=(4,))
# [vx, vy, vz, yaw_rate] — scaled to 5 m/s
# REWARD function:
reward += 1.0 - (cx_err + cy_err) # +1.0 for centred target
reward += min(box_area / total, 1) * 0.5 # +0.5 for being close
reward -= 2.0 # heavy penalty if target lost
```

PPO Training — run separately, not inside `main.py`:

```
model = PPO("MlpPolicy", env, verbose=1,
learning_rate=3e-4, n_steps=2048,
batch_size=64, gamma=0.99,
tensorboard_log="./logs/")
model.learn(total_timesteps=1_000_000)
model.save("models/ppo_hunt_eye")
# Monitor: tensorboard --logdir ./logs/
```

### ■ WARNING

RL may not converge first try. If `ep_rew_mean` doesn't rise after 200k steps, adjust the reward function.

A\* Planner — `ai/planner.py`:

```
# Reads state.cost_map (written by Dev 2 in Phase 2)
# Cells with cost >= 90 = blocked
# 8-directional movement, diagonal cost = sqrt(2) * cell_cost
# Writes output to state.planned_path (Dev 1 renders on dashboard)
```

### ■ MILESTONE

RL policy replaces PID. Drone avoids obstacles. A\* path visible on dashboard.



Final phase — three parallel tracks. Each dev delivers one critical piece, then all three integrate and test together before the real-drone flip.

### Dev 1 Sim-to-Real Switch

Step 15

- Build RealBackend class
- Serial MAVLink instead of UDP
- Camera from real hardware
- HAL routes when mode: real

```
# config.yaml
mode: real
real:
  serial_port: "/dev/ttyUSB0"
  baud_rate: 57600
```

### Dev 2 OSNet Re-ID Engine

Step 13

- 512-dim appearance fingerprint on first lock
  - Cosine similarity match on reappearance
  - Threshold 0.65 — writes to state.active\_target\_id
- ```
sim = dot(emb_a, emb_b) /
(norm(emb_a)*norm(emb_b))
if sim >= 0.65:
  re-lock target
```

### Dev 3 Safety Layer

Step 14

- Last gate before every MAVLink command
  - 5 hard rules — cannot be bypassed
  - Emergency land if any module sets system\_mode = 'emergency'
- ```
checks: heartbeat < 2s
vel <= 5 m/s
alt <= 30m
fence <= 100m
mode != emergency
```

#### Phase 6 — Integration Checklist (all three together):

|             |                                                                                   |
|-------------|-----------------------------------------------------------------------------------|
| Dev 2       | ■ OSNet stores embedding on first target lock                                     |
| Dev 2       | ■ system_mode = "reacquiring" triggers OSNet search                               |
| Dev 3       | ■ Safety layer intercepts all MAVLink sends — no bypass path                      |
| Dev 3       | ■ Emergency land fires when any module sets system_mode = "emergency"             |
| Dev 1       | ■ mode: real in config.yaml → RealBackend loads → serial port opens               |
| ALL         | ■ Run full sim test: target hides → reacquired → geofence triggered → lands       |
| ALL         | ■ Flip to real drone — camera calibration, short hover test, PID retune if needed |
| ■ MILESTONE | FINAL: All 3 integrated. Drone hunts, re-acquires, stays safe. Flip to real.      |

## Appendix — Complete File Ownership Map

| File                          | Owner | Description                                 |
|-------------------------------|-------|---------------------------------------------|
| main.py                       | Dev 1 | Entry point — starts orchestrator           |
| config.yaml                   | Dev 1 | All settings — single source of truth       |
| core/shared_state.py          | Dev 1 | Central whiteboard — all threads read/write |
| core/hal.py                   | Dev 1 | Sim/real hardware switch                    |
| core/orchestrator.py          | Dev 1 | Starts and manages all threads              |
| core/safety.py                | Dev 3 | Last gate — validates every MAVLink command |
| simulation/airsim_client.py   | Dev 1 | AirSim connection wrapper                   |
| simulation/terrain_loader.py  | Dev 2 | GDAL + OSM to cost map                      |
| perception/camera_stream.py   | Dev 1 | 30 Hz frame grabber                         |
| perception/detector.py        | Dev 2 | YOLOv8n inference — writes detections       |
| perception/depth_estimator.py | Dev 2 | MiDaS depth map                             |
| tracking/tracker.py           | Dev 2 | ByteTrack multi-object tracker              |
| tracking/target_filter.py     | Dev 2 | Selective lock-on + Re-ID trigger           |
| tracking/reid_engine.py       | Dev 2 | OSNet 512-dim fingerprint matching          |
| ai/gym_env.py                 | Dev 3 | Gymnasium RL environment wrapper            |
| ai/agent.py                   | Dev 3 | PPO policy load + inference                 |
| ai/planner.py                 | Dev 3 | A* path planner on cost map                 |
| control/mavlink_interface.py  | Dev 3 | MAVLink sends + PID controller              |
| control/real_backend.py       | Dev 1 | Serial MAVLink for real drone               |
| ui/dashboard.py               | Dev 1 | PyQt6 live monitoring window                |

## Recommended Git Workflow

```
# Each dev works on their own branch:
git checkout -b dev1/infra
git checkout -b dev2/vision
git checkout -b dev3/ai-control
# Merge to main after each phase milestone:
git checkout main && git merge dev1/infra
# Never push directly to main – PR + one other dev reviews
```

**Hunt Eye · 3 devs · 15 steps · One drone that hunts.**